Лабораторная работа

Генераторы псевдослучайных чисел

1. Линейный конгруэнтный метод

Данный метод был разработан Лехмером и стал первой, широко используемой, технологией генерации случайного числа.

Рисунок 1 показывает работу этого метода, который рекурсивно создает последовательность псевдослучайных чисел, используя линейное конгруэнтное уравнение $x_{i+1} = (ax_i + b) mod\, n$, где x_0 называется *начальным числом* (*seed*) – это число между 0 и (n – 1).

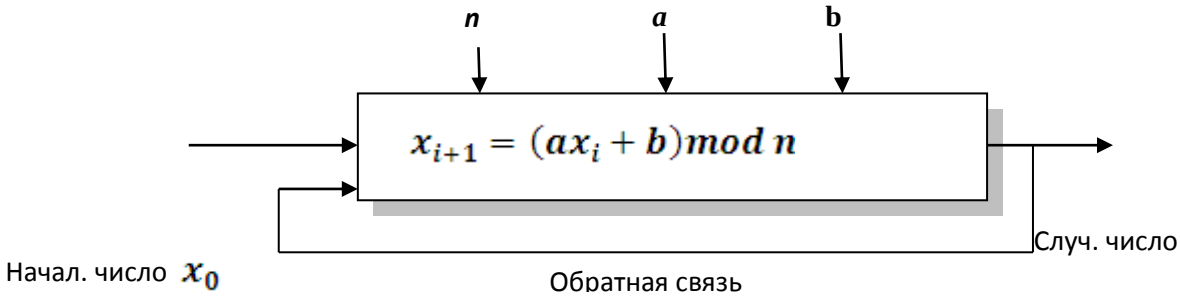

Рисунок 1

Последовательность является периодической, где период зависит от того, как тщательно выбраны коэффициенты a и b. Идеально период должен быть такого размера, как модуль n.

Проиллюстрируем работу этого алгоритма на примере.

Как мы упоминали выше, в линейном конгруэнтном методе для вычисления k_i используем формулу

$$k_i = (a * k_{i-1} + b) mod\, c.$$

Для получения k_i задается начальное значение k_0. Возьмем в качестве примера $a = 5; b = 3; c = 11, k_0 = 1$. В этом случае по приведеной выше формуле получим значения от 0 до 10, т. к. $c = 11$. Вычислим несколько элементов последовательности:

$$k_1 = (5 * 1 + 3) mod\, 11 = 8,$$
$$k_2 = (5 * 8 + 3) mod\, 11 = 10,$$
$$k_3 = (5 * 10 + 3) mod\, 11 = 9,$$
$$k_4 = (5 * 9 + 3) mod\, 11 = 4,$$
$$k_5 = (5 * 4 + 3) mod\, 11 = 1.$$

Полученные значения (8, 10, 9, 4, 1) выглядят похожими на случайные числа. Однако, следующее значение k_6 снова будет равно 8:

$$k_6 = (5 * 1 + 3) mod\, 11 = 8,$$

а значения k_7 и k_8 будут равны 10 и 9 соответственно.

Получается, что наш генератор псевдослучайных чисел повторяется, порождая периодически числа 8, 10, 9, 4, 1.

К сожалению, это свойство характерно для всех линейных конгруэнтных генераторов. Изменяя значения основных параметров *a*, *b* и *c*, можно влиять на длину периода и на сами порождаемые значения k_i.

Увеличение числа *c* в общем случае ведет к увеличению периода. Если параметры *a*, *b* и *c* выбраны правильно, то генератор будет порождать случайные числа с максимальным периодом, равным *c*.

Достоинством линейных конгруэнтных генераторов псевдослучайных чисел является их простота и высокая скорость получения псевдослучайных значений. Линейные конгруэнтные генераторы находят применение при решении задач моделирования и математической статистики, однако в криптографических целях их нельзя рекомендовать к использованию, так как специалисты по криптоанализу научились восстанавливать всю последовательность ПСЧ по нескольким значениям.

Алгоритм параметризуется четырьмя числами следующим образом (Табл. 1):

Таблица 1. Линейный конгруэнтный метод. Параметризация чисел

m	Модуль (основание системы)	$m > 0$
a	Множитель	$0 \le a < m$
c	Приращение	$0 \le c < m$
$x0$	Начальное значение или зерно (seed)	$0 \le x0 < m$

Последовательность $\{X_n\}$ получается следующим образом:

$X_{n+1} = (aX_n + c)\ mod\ m.$

Если m, a и c являются целыми числами, то X_n – последовательность целых чисел, которая задается в диапазоне $0 \le X_n < m$.

Выбор значений для m, a и c является критическим для разработки хорошего генератора случайных чисел.

m должно быть очень большим, чтобы была возможность создать много случайных чисел. Считается, что *m* должно быть приблизительно равно максимальному положительному целому числу для данного компьютера.

Существует три критерия, используемые при выборе генератора случайных чисел:

- Функция должна создавать полный период.
- Создаваемая последовательность должна появляться случайно. Последовательность не является случайной, так как она создается детерминированно, но различные статистические тесты, которые

могут применяться, должны показывать, что последовательность случайна.

- Функция должна эффективно реализовываться на 32-битных процессорах.

Значения *a, c* и *m* должны быть выбраны таким образом, чтобы эти три критерия выполнялись. Для 32-битной арифметики соответствующее простое значение $m = 2^{31} - 1$. Таким образом, функция создания псевдослучайных чисел имеет вид:

$$x_{n+1} = (ax_n)\ mod\ (2^{31} - 1).$$

Только небольшое число значений *a* удовлетворяет всем трем критериям. Одно из таких значений есть *a* = 7^5 = 16807, которое использовалось в семействе компьютеров IBM 360. Этот генератор широко применяется и прошел более тысячи тестов.

Если ***Z*** – основание системы счисления, которое используется в машине, ***e*** – разрядность, ***m*** – размер слова, $\boldsymbol{m = z^e}$, то

$$a = z^k + 1,\ \ 2 \le k \le e,$$

тогда

$$x_{n+1} = ((z^k + 1) * x_n)\ mod\ z^e$$

Если криптоаналитику известны значения x_0, x_1, x_2, x_3, то они удовлетворяют системе уравнений:

$$\begin{cases} x_1 = (a * x_0 + c) mod\ m, \\ x_2 = (a * x_2 + c) mod\ m, \\ x_3 = (a * x_3 + c) mod\ m, \end{cases}$$

из которых могут быть получены параметры *a, c* и *m*.

Введем некоторые рекомендации, которые позволят улучшить данный алгоритм.

1. Оптимальный выбор модуля, *n* , — это наибольшее простое число, близкое к размеру слова, используемого в компьютере. Рекомендуется использовать тридцать первое простое число Мерсенна, как модуль $n = M_{31} = 2^{31} - 1$.
2. Чтобы создавать период, равный значению модуля, значение первого коэффициента, *a*, должно быть первообразным корнем главного модуля. Хотя целое число 7 - первообразный корень M_{31}, рекомендуют использовать 7^k, где *k* - целое число, взаимно-простое с ($M_{31} - 1$). Некоторые рекомендованные значения для *k* это 5 и 13. Это означает, что ($a = 7^5$) или ($a = 7^{13}$).
3. Для эффективного использования компьютера значение второго коэффициента *b* должно быть нулевым.

Задание. *Используя данные рекомендации, улучшить программную реализацию.*

2. Метод Фибоначчи с запаздыванием

Один из широко распространённых фибоначчиевых датчиков основан на следующей итеративной формуле:

$$X_k = \begin{cases} X_{k-a} - X_{k-b}, & \text{если } X_{k-a} \geq X_{k-b}; \\ X_{k-a} - X_{k-b} + 1, & \text{если } X_{k-a} < X_{k-b}; \end{cases}$$

где X_k — вещественные числа из диапазона [0,1), *a*, *b* — целые положительные числа, называемые лагами.

При реализации через целые числа достаточно формулы

$$X_k = X_{k-a} - X_{k-b}.$$

Для работы фибоначчиеву датчику требуется знать *max{a,b}* предыдущих сгенерированных случайных чисел. При программной реализации для хранения сгенерированных случайных чисел используется конечная циклическая очередь на базе массива. Для старта фибоначчиевому датчику требуется *max{a,b}* случайных чисел, которые могут быть сгенерированы простым конгруэнтным датчиком.

Период фибоначчиева датчика может быть оценен по следующей формуле:

$$T = \left(2^{\max\{a,b\}} - 1\right) * 2^l,$$

где *l* – число битов в мантиссе вещественного числа.

Приведем пример того, как работает данный алгоритм.

Вычислим последовательность из первых десяти чисел, генерируемую методом Фибоначчи с запаздыванием, начиная с k_5 при следующих исходных
данных: $a = 4, b = 1, k_0 = 0.1, k_1 = 0.7, k_2 = 0.3, k_3 = 0.9, k_4 = 0.5$:

$$k_5 = k_1 - k_4 = 0.7 - 0.5 = 0.2;$$
$$k_6 = k_2 - k_5 = 0.3 - 0.2 = 0.1;$$
$$k_7 = k_3 - k_6 = 0.9 - 0.1 = 0.8;$$
$$k_8 = k_4 - k_7 + 1 = 0.5 - 0.8 + 1 = 0.7;$$
$$k_9 = k_5 - k_8 + 1 = 0.2 - 0.7 + 1 = 0.5;$$
$$k_{10} = k_6 - k_9 + 1 = 0.1 - 0.5 + 1 = 0.6;$$
$$k_{11} = k_7 - k_{10} = 0.8 - 0.6 = 0.2;$$
$$k_{12} = k_8 - k_{11} = 0.7 - 0.2 = 0.5;$$
$$k_{13} = k_9 - k_{12} + 1 = 0.5 - 0.5 + 1 = 1;$$
$$k_{14} = k_{10} - k_{13} + 1 = 0.6 - 1 + 1 = 0.6.$$

Видим, что генерируемая последовательность чисел внешне похожа на случайную. Действительно, исследования подтверждают, что получаемые случайные числа обладают хорошими статистическими свойствами.

В литературных источниках указывается, что данный метод обладает хорошей криптостойкостью и может использоваться в криптографии.

ГПСЧ на основе данного алгоритма применяется в математических и статических расчетах, а также при моделировании случайных процессов.

Задание. *Реализовать Метод Фибоначчи с запаздыванием.*

3. Алгоритм Блюма-Блюма-Шуба

Это генератор псевдослучайных чисел, предложенный в 1986 году Ленор Блюм, Мануэлем Блюмом и Майклом Шубом.

Он выглядит следующим образом:

$x_{n+1} = x_n^2 \bmod M,$

где $M = pq$ – произведение двух простых чисел.

Данный алгоритм заключается в следующем. Вначале выбираются два больших простых числа *p* и *q*. Числа *p* и *q* должны быть оба сравнимы с 3 по модулю 4, то есть при делении *p* и *q* на 4 должен получаться одинаковый остаток 3. Далее вычисляется число *M* = *pq*, называемое *целым числом Блюма*. Затем выбирается другое случайное целое число x, взаимно простое с M. Вычисляем $x_0 = x^2 \bmod M$. x_0 называется *стартовым числом генератора*.

На каждом *n*-м шаге работы генератора вычисляется $x_{n+1} = x_n^2 \bmod M$. Результатом *n*-го шага является один бит числа x_{n+1}.

Иногда, в качестве результата принимают бит чётности, то есть количество единиц в двоичном представлении элемента. Если количество единиц в записи числа четное – бит четности принимается равным 0, нечетное – бит четности принимается равным 1.

Схематично работу BBS генератора можно представим следующим образом. (Рис. 2)

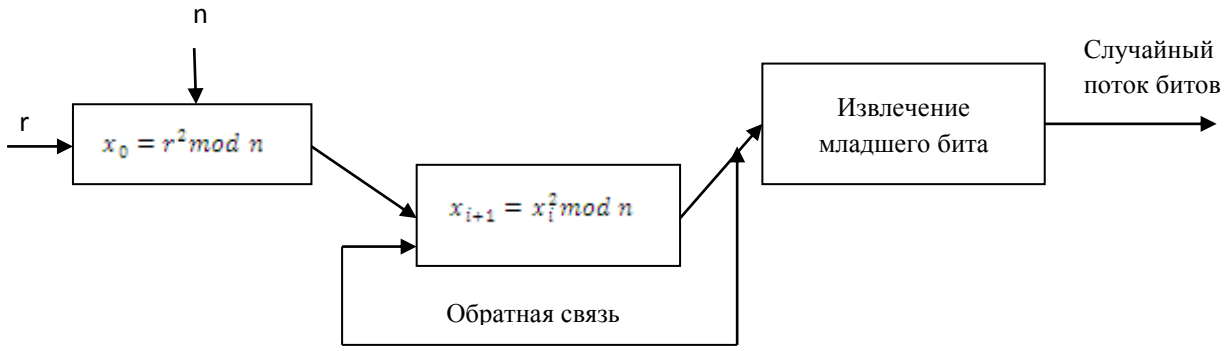

Рисунок 2

Рассмотрим на примере, как работает данный алгоритм.

Пусть *p* = 11, *q* = 19. Убедимся, что $11 \bmod 4 = 3, 19 \bmod 4 = 3$. Найдем произведение $M = pq = 11 * 19 = 209$. Пусть $x = 3$. Вычислим x_0.

$x_0 = 3^2 \bmod 209 = 9.$

Вычислим первые десять чисел x_i по алгоритму BBS. В качестве случайных бит будем брать младший бит в двоичной записи числа x_i:

$x_1 = 9^2 mod\ 209 = 81\ mod\ 209 = 81,$ младший бит 1,

$x_2 = 81^2 mod\ 209 = 6561\ mod\ 209 = 82,$ младший бит 0,

$x_3 = 82^2 mod\ 209 = 6724\ mod\ 209 = 36,$ младший бит 0,

$x_4 = 36^2 mod\ 209 = 1296\ mod\ 209 = 42,$ младший бит 0,

$x_5 = 42^2 mod\ 209 = 1764\ mod\ 209 = 92,$ младший бит 0,

$x_6 = 92^2 mod\ 209 = 8464\ mod\ 209 = 104,$ младший бит 0,

$x_7 = 104^2 mod\ 209 = 10816\ mod\ 209 = 157,$ младший бит 1,

$x_8 = 157^2 mod\ 209 = 24649\ mod\ 209 = 196,$ младший бит 0,

$x_9 = 196^2 mod\ 209 = 38416\ mod\ 209 = 169,$ младший бит 1,

$x_{10} = 169^2 mod\ 209 = 28561\ mod\ 209 = 137,$ младший бит 1.

Самым интересным для практических целей свойством этого метода является то, что для получения *n*-го числа последовательности не нужно вычислять все предыдущие *n* чисел x_i. Оказывается x_n можно сразу получить по формуле

$$x_n = x_0^{2^n mod\ ((p-1)(q-1))} mod\ M.$$

Например, вычислим x_{10} сразу из x_0:

$$x_{10} = x_0^{2^{10} mod((11-1)(19-1))} mod\ 209 = 9^{124} mod\ 209 = 81^{31} mod\ 209$$
$$= (144 * 104) mod\ 209 = 14976\ mod\ 209 = 137.$$

В результате действительно получили такое же значение, как и при последовательном вычислении, – 137. Вычисления кажутся достаточно сложными, однако на самом деле их легко оформить в виде небольшой процедуры или программы и использовать при необходимости.

Интересной особенностью этого алгоритма является то, что для получения x_n необязательно вычислять все *n − 1* предыдущих чисел, если известно начальное состояние генератора x_0 и числа *p* и *q*. *n*-ное значение может быть вычислено «напрямую» используя формулу:

$$x_n = x_0^{2^n\ mod\ ((p-1)(q-1))}\ mod\ M.$$

Представим трассировку программной реализации данного метода.

Пусть первоначально X = 2701, тогда будем иметь следующую таблицу (См. табл. 2)

Табл. 2

t	p	q	M	Операции	X
0	2047	8191	16766977	$(x * x) mod\ M = 7295401\ mod\ M$	7295401
1	2047	8191	16766977	$53222875750801\ mod\ M$	13969942

2	2047	8191	16766977	$195159279483364 \bmod M$	390933
3	2047	8191	16766977	$152828610489 \bmod M$	14382111
4	2047	8191	16766977	$206845116816321 \bmod M$	9268855
5	2047	8191	16766977	$85911673011025 \bmod M$	13472828
6	2047	8191	16766977	$181517094317584 \bmod M$	14556548
7	2047	8191	16766977	$211893089676304 \bmod M$	15431356
8	2047	8191	16766977	$238126747998736 \bmod M$	11238657
9	2047	8191	16766977	$126307411163649 \bmod M$	12890064

Задание.

1. *Реализовать метод Блюма-Блюма-Шуба.*
2. *Провести сравнительную характеристику трех рассмотренных методов генерации псевдослучайных чисел (скорость, длина периода, закон распределения).*